Software Requirements Specification (SRS)

Project: Web-Based ML Model Evaluator

Version: 1.0

Authors: < vasanth kumar , munireddy,manoj,sharath>

Date: 04-09-2026

Status: Draft / Academic Submission

Programming Stack: Python (Django) / JavaScript (React or equivalent)

Machine Learning: Scikit-learn, Pandas, NumPy

Revision history

Version	Date	Author	Change summary
1.0	04-09-2026	Instructor / Student	Initial SRS with requirements, acceptance criteria, security requirements, UML/use-case structure, and RTM.

Approvals

Role	Name	Signature / Email	Date
Course Coordinator	ASHA K A		
Project Guide	ASHA K A		
Team Lead / Student	VASANTH KUMAR REDDY		

Web-Based ML Model Evaluator
Software Requirements Specification

Table of Contents

1. Introduction

2. Overall description

3. External interfaces

4. System features (detailed)

5. Non-functional requirements (detailed)

6. Quality attributes & Acceptance tests

7. UML Use-Case Diagram

8. Requirements Traceability Matrix (RTM)

1. Introduction

1.1 Purpose

This document is a Software Requirements Specification (SRS) for a Web-Based ML Model Evaluator. It defines the functional and non-functional requirements, interfaces, security objectives, verification criteria, and acceptance conditions for a web application that supports an end-to-end machine-learning experimentation workflow.

1.2 Scope

The system covers dataset upload and validation, dataset inspection, data preprocessing, feature and target selection, classification and regression model selection, model configuration, training, testing, evaluation, model comparison, result visualization, experiment history, and result export. The initial release focuses on classical supervised machine-learning workflows and does not require unrestricted execution of user-supplied code.

1.3 Audience

The intended audience includes developers, QA engineers, ML/data-science students, researchers, project guides, system administrators, and assessment evaluators.

1.4 Definitions

Term	Definition
ML	Machine Learning
SRS	Software Requirements Specification
API	Application Programming Interface
UI	User Interface
CSV	Comma-Separated Values
REST	Representational State Transfer
MAE	Mean Absolute Error
MSE	Mean Squared Error
RMSE	Root Mean Squared Error
ROC-AUC	Receiver Operating Characteristic – Area Under Curve
RTM	Requirements Traceability Matrix

2. Overall description

2.1 Product perspective

The Web-Based ML Model Evaluator is a browser-accessible application consisting of a web interface, Django backend/API layer, Python-based data and machine-learning processing services, and a relational database. The system coordinates the flow from dataset upload through preprocessing, model training/testing, evaluation, and comparison.

Recommended architecture:

Layer	Main responsibility
Web UI	Dataset upload, configuration, progress display, results, and visualizations.
Django / REST API	Authentication, request validation, orchestration, and access control.
Data Pipeline	Loading, profiling, cleaning, encoding, scaling, and train-test splitting.
ML Engine	Model configuration, training, prediction, and evaluation.
Persistence	Users, dataset metadata, experiments, configurations, metrics, and optional model artifacts.

2.2 Major product functions (detailed)

- Authenticate users and maintain protected sessions.
- Upload and validate CSV datasets.
- Preview dataset structure and basic statistics.
- Detect and handle missing values and duplicate records.
- Encode categorical variables and scale numerical features.
- Select input features and target variable.
- Support classification and regression workflows.
- Select and configure machine-learning algorithms.
- Train and test selected models.
- Calculate appropriate evaluation metrics.
- Compare multiple model experiments.
- Visualize evaluation results.
- Store experiment history and configurations.
- Export results and optionally save model artifacts.
- Provide administrative management of users and stored resources.

2.3 User roles and characteristics (expanded)

Role	Characteristics / Responsibilities
Normal User	Basic ML knowledge; uploads datasets, configures pipelines, trains models, evaluates and compares results.
Administrator	Manages users and stored resources, monitors activity, and removes invalid or unwanted resources.
Assessment Evaluator	Reviews requirements, workflow, results, acceptance tests, and traceability evidence.

2.4 Operating environment

The system shall operate through current web browsers such as Chrome, Firefox, Edge, or equivalent. The backend may run on Linux or Windows with Python, Django, Scikit-learn, and a supported relational database. A production deployment should use HTTPS.

2.5 Constraints

- Initial version primarily supports supervised learning: classification and regression.
- CSV is the primary dataset format for the first release.
- Training capacity is limited by available CPU, RAM, storage, and deployment resources.
- The system shall not execute arbitrary user-supplied Python code.
- Some algorithms require numerical input and therefore depend on suitable preprocessing.
- The system does not guarantee that the selected model is scientifically optimal for every dataset.

3. External interface requirements

3.1 User interfaces

- Login and registration interface with clear validation messages.
- Dashboard showing uploaded datasets and recent experiments.
- Dataset upload and preview interface.
- Data preprocessing and feature/target selection interface.
- Model selection and hyperparameter configuration interface.
- Training-progress interface with status feedback.
- Results dashboard with metrics, charts, confusion matrix/ROC where applicable, and model comparison.
- Experiment history and result export interface.
- Accessibility: keyboard-accessible controls, readable contrast, labels for form controls, and responsive layout.

3.2 Hardware interfaces

The initial system does not require specialized hardware. It may run on standard development/server hardware. Optional future deployments may use GPUs for deep-learning extensions.

3.3 Software interfaces

Interface	Purpose
Django / Django REST Framework	Web application and API services.
Scikit-learn	Classical ML algorithms, preprocessing utilities, and evaluation functions.
Pandas / NumPy	Dataset loading, manipulation, numerical computation, and profiling.
PostgreSQL / MySQL	Persistence of users, metadata, experiments, configurations, and metrics.
Plotly / Chart.js / Matplotlib	Result visualization and chart generation.
Git / GitHub	Source-code version control and collaboration.

3.4 Communications

- HTTPS/TLS shall be used for production network communication.
- Frontend-to-backend communication shall use authenticated HTTP(S) requests/API calls.
- API requests shall validate input and return controlled error responses.
- Long-running training operations should be designed to support asynchronous job processing in future versions.

4. System features (detailed)

Each requirement below includes an acceptance criterion and a reference test case. IDs follow ML-EVAL-F-###.

4.1 Authentication & User Management

Req ID	Requirement (shall...)	Type	Priority	Source / Stakeholder	Acceptance criteria / Test case ref	Comments / Dependencies
ML-EVAL-F-001	The system shall allow a new user to register with valid account details.	Functional	High	User	Valid registration creates an account. Test: TC-AUTH-01	Database and validation required.
ML-EVAL-F-002	The system shall authenticate registered users and restrict protected functions to authenticated users.	Functional	High	Security / User	Valid login grants access; invalid login is rejected. Test: TC-AUTH-02	Authentication module required.
ML-EVAL-F-003	The system shall provide logout and shall invalidate the authenticated session/token.	Functional	Medium	User / Security	After logout, protected request is rejected. Test: TC-AUTH-03	Depends on authentication mechanism.

4.2 Dataset Management & Data Pipeline

Req ID	Requirement (shall...)	Type	Priority	Source / Stakeholder	Acceptance criteria / Test case ref	Comments / Dependencies
ML-EVAL-F-010	The system shall allow authenticated users to upload a supported CSV dataset.	Functional	High	User	Valid CSV is accepted and metadata is created. Test: TC-DATA-01	File validation/storage required.
ML-EVAL-F-011	The system shall validate file type, size, structure, and readable tabular content before processing.	Functional	High	Security / Data	Invalid file is rejected with a meaningful message. Test: TC-DATA-02	Secure upload handling required.
ML-EVAL-F-012	The system shall display dataset preview, row/column counts, data types, and	Functional	High	User	Preview matches uploaded data. Test: TC-DATA-03	Pandas/data-profiling layer.

	missing-value summary.					
ML-EVAL-F-013	The system shall provide preprocessing options for missing values and duplicate records.	Functional	High	ML User	Selected cleaning method is applied to the training pipeline. Test: TC-DATA-04	Preprocessing pipeline required.
ML-EVAL-F-014	The system shall support categorical encoding and numerical feature scaling where required.	Functional	High	ML User	Configured transformations are applied without data-type errors. Test: TC-DATA-05	Scikit-learn preprocessing.
ML-EVAL-F-015	The system shall allow the user to select input features, target variable, and train-test split ratio.	Functional	High	ML User	Selected X/y and split configuration are used for training. Test: TC-DATA-06	Target must be valid.

4.3 Model Selection & Configuration

Req ID	Requirement (shall...)	Type	Priority	Source / Stakeholder	Acceptance criteria / Test case ref	Comments / Dependencies
ML-EVAL-F-020	The system shall support classification and regression problem types.	Functional	High	ML User	Both workflows can be selected and executed. Test: TC-ML-01	Model registry required.
ML-EVAL-F-021	The system shall provide suitable classification algorithms including Logistic Regression, Decision Tree, Random Forest, KNN, SVM, and Naive Bayes.	Functional	High	ML User	Each listed model can be selected on compatible data. Test: TC-ML-02	Scikit-learn.
ML-EVAL-F-022	The system shall provide suitable regression algorithms including	Functional	High	ML User	Each listed model can be selected on compatible data. Test: TC-ML-03	Scikit-learn.

	Linear Regression, Decision Tree Regressor, Random Forest Regressor, SVR, and Gradient Boosting.					
ML-EVAL-F-023	The system shall allow users to configure supported model hyperparameters and shall provide defaults.	Functional	Medium	ML User	Changed parameter is reflected in experiment configuration. Test: TC-ML-04	Validation required.

4.4 Training, Testing & Evaluation

Req ID	Requirement (shall...)	Type	Priority	Source / Stakeholder	Acceptance criteria / Test case ref	Comments / Dependencies
ML-EVAL-F-030	The system shall train the selected model using the configured training dataset and preprocessing pipeline.	Functional	High	ML User	Training completes and produces a fitted model or controlled error. Test: TC-TRAIN-01	ML engine required.
ML-EVAL-F-031	The system shall generate predictions for the test dataset using the trained model.	Functional	High	ML User	Prediction count matches test-set records. Test: TC-TRAIN-02	Requires fitted model.
ML-EVAL-F-032	For classification, the system shall calculate accuracy, precision, recall, F1-score, and confusion matrix; ROC-AUC where applicable.	Functional	High	Evaluator / ML User	Metrics are generated correctly for a known test dataset. Test: TC-EVAL-01	Metric selection depends on problem type.
ML-EVAL-F-033	For regression, the system shall calculate MAE, MSE, RMSE, and	Functional	High	Evaluator / ML User	Metrics match reference calculations within defined tolerance. Test: TC-	Metric selection depends on problem type.

	R^2.				EVAL-02	
ML-EVAL-F-034	The system shall provide clear training status and report training failures without crashing the application.	Functional	High	User / Reliability	Failure produces a controlled message and recorded error. Test: TC-TRAIN-03	UI and backend error handling.

4.5 Comparison, History & Export

Req ID	Requirement (shall...)	Type	Priority	Source / Stakeholder	Acceptance criteria / Test case ref	Comments / Dependencies
ML-EVAL-F-040	The system shall allow users to compare results from at least two compatible model experiments.	Functional	High	Evaluator / User	Comparison table displays selected metrics for both models. Test: TC-COMP-01	Requires experiment history.
ML-EVAL-F-041	The system shall visualize model evaluation results using suitable charts and tables.	Functional	Medium	User / Evaluator	Charts reflect stored metric values. Test: TC-COMP-02	Plotting library required.
ML-EVAL-F-042	The system shall store experiment metadata, model configuration, preprocessing configuration, and evaluation results.	Functional	High	Evaluator / User	Experiment can be reopened with stored results. Test: TC-HIST-01	Database required.
ML-EVAL-F-043	The system shall allow users to export evaluation results in at least one supported machine-readable format.	Functional	Medium	User	Exported file contains selected experiment results. Test: TC-EXP-01	CSV/JSON recommended.

5. Non-functional requirements (detailed)

NFRs below are measurable where practical and are tied to test plans. IDs follow ML-EVAL-NF-###.

Req ID	Requirement	Category	Priority	Acceptance criteria / Measurement
ML-EVAL-NF-001	For routine dataset metadata operations, the system should return a response within 3 seconds under normal test load.	Performance	High	90th percentile ≤ 3 s in production-like test. Test: TC-PERF-01
ML-EVAL-NF-002	The system shall provide clear progress/status feedback for model-training operations that exceed normal interactive response time.	Usability / Performance	High	Training state is visible until completion/failure. Test: TC-UX-01
ML-EVAL-NF-003	The application shall be designed for reliable operation and shall handle invalid inputs without uncontrolled termination.	Reliability	High	Negative test suite produces controlled responses. Test: TC-REL-01
ML-EVAL-NF-004	The application shall enforce access control so users cannot access other users' private datasets or experiments.	Security	High	Unauthorized access tests fail with appropriate response. Test: TC-SEC-01
ML-EVAL-NF-005	The user interface shall support readable, responsive, and keyboard-accessible interactions.	Usability / Accessibility	Medium	Accessibility checklist passes for primary workflow. Test: TC-UX-02

5.1. Security

5.1.1 Security Objectives

- Confidentiality: protect user accounts, datasets, experiment records, and stored model artifacts from unauthorized access.
- Integrity: prevent unauthorized modification of datasets, configurations, results, and experiment records.
- Safe processing: validate uploaded files and never execute arbitrary user-supplied code as part of the ML workflow.

5.1.2 Security Requirements

Req ID	Requirement (shall...)	Type	Priority	Acceptance criteria / Test case ref
ML-EVAL-SR-001	TLS 1.2+ shall be used for production network connections, with HTTPS enforced.	Security	High	HTTPS connection verified. Test: TC-SEC-02
ML-EVAL-SR-002	Passwords shall be securely hashed and shall never be stored in plaintext.	Security	High	Database inspection confirms no plaintext passwords. Test: TC-SEC-03
ML-EVAL-SR-003	The system shall validate uploaded file type and size and shall reject unsupported or unsafe uploads.	Security	High	Malicious/ unsupported file tests are rejected. Test: TC-SEC-04
ML-EVAL-SR-004	The system shall enforce authorization checks for dataset, experiment, and model-artifact access.	Security	High	Cross-user access attempt is denied. Test: TC-SEC-05
ML-EVAL-SR-005	The system shall not execute arbitrary Python code supplied through datasets, model parameters, or user input.	Security	High	Code-injection test does not execute payload. Test: TC-SEC-06
ML-EVAL-SR-006	The system shall avoid exposing internal stack traces or sensitive configuration details in user-facing error messages.	Security	Medium	Forced server error returns sanitized message. Test: TC-SEC-07

6. Quality attributes & Acceptance tests

6.1 Quality attributes

Attribute	Expected quality
Performance	Responsive metadata operations and visible progress for long ML tasks.
Reliability	Controlled handling of invalid inputs and model-training failures.
Security	Authentication, authorization, secure file handling, encrypted transport, and safe processing.
Usability	Simple step-by-step workflow with clear labels and meaningful feedback.
Maintainability	Modular separation of UI, API, data pipeline, ML engine, and persistence.
Scalability	Architecture permits background workers and scalable storage in future releases.
Portability	Deployable on common Linux/Windows server environments.

6.2 Acceptance test suites

Suite	Representative tests
Authentication	TC-AUTH-01 to TC-AUTH-03
Dataset / Data Pipeline	TC-DATA-01 to TC-DATA-06
Model Selection	TC-ML-01 to TC-ML-04
Training & Testing	TC-TRAIN-01 to TC-TRAIN-03
Evaluation	TC-EVAL-01 to TC-EVAL-02
Comparison & History	TC-COMP-01 to TC-COMP-02, TC-HIST-01
Export	TC-EXP-01
Performance	TC-PERF-01
Security	TC-SEC-01 to TC-SEC-07
Accessibility / UX	TC-UX-01 to TC-UX-02

6.3 Exit criteria for acceptance

All high-priority functional requirements shall be implemented and verified, no critical security or reliability failures shall remain, and the RTM shall show traceability from requirements to design modules and test cases.

7. UML Use-Case Diagram

The UML use-case diagram below defines the complete scope of user and administrative interactions supported by the Web-Based ML Model Evaluator. The system boundary encapsulates the end-to-end machine-learning experimentation workflow, spanning account authentication, dataset upload and validation, preprocessing configuration, model selection, training execution, multi-model evaluation, and result export.

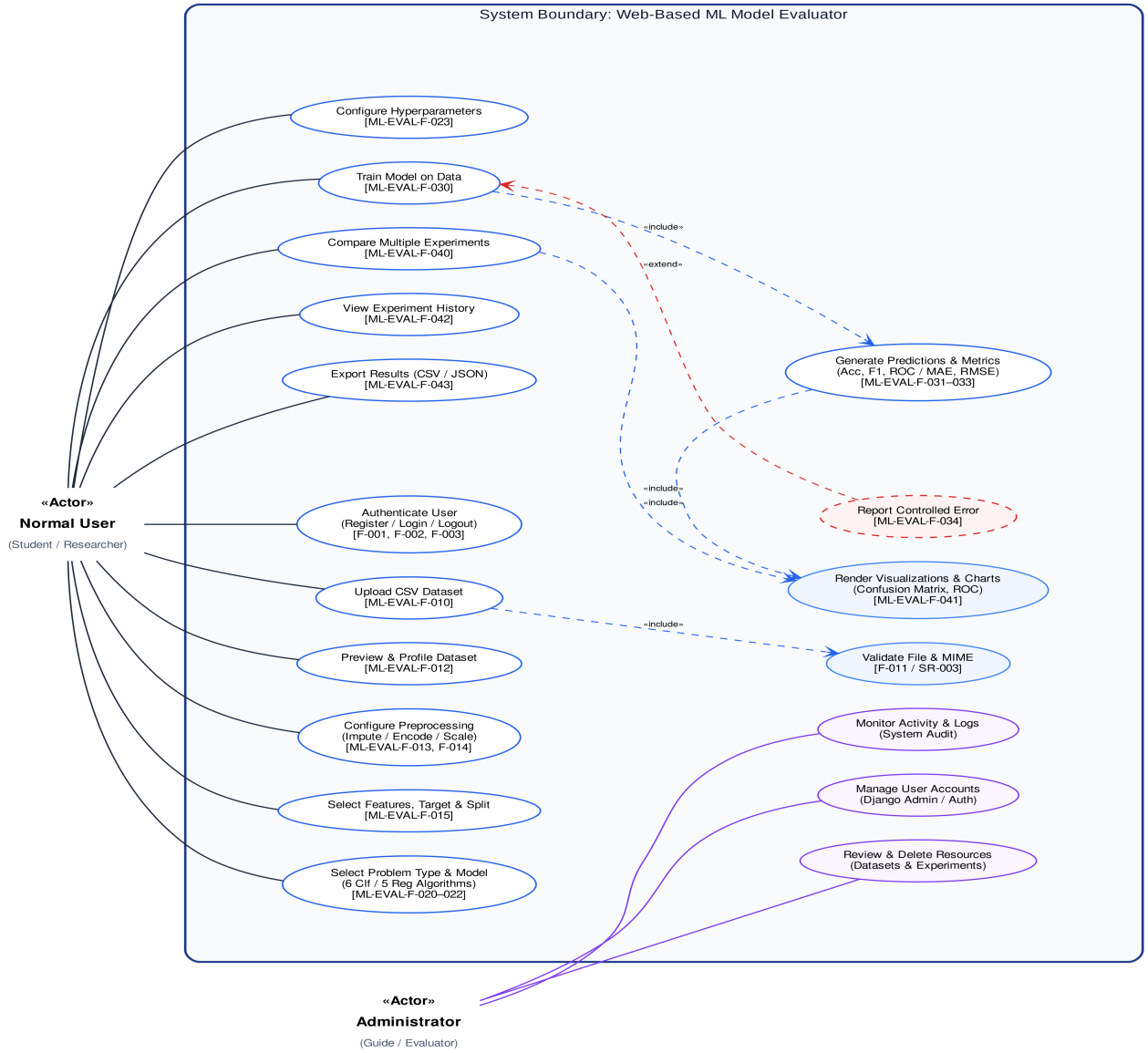


Figure 1. *UML Use-Case Diagram for Web-Based ML Model Evaluator (SRS ML-EVAL-F-001 to ML-EVAL-F-043).*

7.1 Use-Case Diagram 1 – User ML Evaluation Workflow

Primary Actor: Normal User (Student / Researcher / ML Practitioner)

Workflow: Normal User → Register / Login → Upload Dataset → Validate / Preview Dataset → Configure Preprocessing → Select Features & Target → Select Problem Type → Select Model → Configure Hyperparameters → Train Model → Test Model → View Metrics → Compare Models → Export Results.

7.2 Use-Case Diagram 2 – Administration & Experiment Management

Actors: Normal User, Administrator

Normal User Flow: Normal User → View Experiment History → Reopen Results → Delete Own Dataset/Experiment → Export Results.

Administrator Flow: Administrator → Manage Users → Review Stored Resources → Remove Invalid/Unwanted Resources → Monitor System Activity.

7.3 System Architecture Diagram Flow

Browser / Responsive Web UI → Django REST API → Data Pipeline + ML Engine → Database / File Storage. The API layer handles authentication and authorization; the Data Pipeline handles preprocessing and feature preparation; the ML Engine handles model training/testing/evaluation; persistence stores metadata, results, and optional model artifacts.

7.4 Suggested Activity Flow

Start → Login → Upload Dataset → Validate → Preview → Preprocess → Select Features/Target → Select Classification/Regression → Select Model → Configure Parameters → Train → Test → Evaluate → Compare / Export → End.

8. Requirements Traceability Matrix (RTM)

Status values: N = Not started, P = Partially implemented, A = Accepted/verified. Update these values during implementation and testing.

Req ID	Requirement short	Section ref / Design Spec	Module	Test case(s)	Status (N/P/A)
ML-EVAL-F-001	User registration	4.1 / DS-AUTH-01	AuthModule	TC-AUTH-01	N
ML-EVAL-F-002	User authentication	4.1 / DS-AUTH-02	AuthModule	TC-AUTH-02	N
ML-EVAL-F-010	Upload CSV dataset	4.2 / DS-DATA-01	DatasetModule	TC-DATA-01	N
ML-EVAL-F-011	Validate uploaded dataset	4.2 / DS-DATA-02	DatasetModule	TC-DATA-02	N
ML-EVAL-F-012	Preview and profile dataset	4.2 / DS-DATA-03	DataPipeline	TC-DATA-03	N
ML-EVAL-F-013	Handle missing values/duplicates	4.2 / DS-DATA-04	PreprocessingModule	TC-DATA-04	N
ML-EVAL-F-015	Select features, target, split	4.2 / DS-DATA-06	PipelineConfig	TC-DATA-06	N
ML-EVAL-F-020	Classification/ regression workflow	4.3 / DS-ML-01	ModelSelector	TC-ML-01	N
ML-EVAL-F-021	Classification models	4.3 / DS-ML-02	ModelRegistry	TC-ML-02	N
ML-EVAL-F-022	Regression models	4.3 / DS-ML-03	ModelRegistry	TC-ML-03	N
ML-EVAL-F-023	Hyperparameter configuration	4.3 / DS-ML-04	ModelConfig	TC-ML-04	N
ML-EVAL-F-030	Train selected model	4.4 / DS-TRAIN-01	TrainingEngine	TC-TRAIN-01	N
ML-EVAL-F-031	Generate test predictions	4.4 / DS-TRAIN-02	TestingEngine	TC-TRAIN-02	N
ML-EVAL-F-032	Classification metrics	4.4 / DS-EVAL-01	EvaluationEngine	TC-EVAL-01	N
ML-EVAL-F-033	Regression metrics	4.4 / DS-EVAL-02	EvaluationEngine	TC-EVAL-02	N
ML-EVAL-F-040	Compare experiments	4.5 / DS-COMP-01	ComparisonModule	TC-COMP-01	N
ML-EVAL-F-041	Visualize results	4.5 / DS-COMP-02	VisualizationModule	TC-COMP-02	N
ML-EVAL-F-042	Store experiment history	4.5 / DS-HIST-01	ExperimentModule	TC-HIST-01	N
ML-EVAL-F-043	Export results	4.5 / DS-EXP-01	ExportModule	TC-EXP-01	N
ML-EVAL-NF-001	Routine response-time target	5 / DS-PERF-01	WebUI/API	TC-PERF-01	N
ML-EVAL-NF-004	Cross-user access control	5.1 / DS-SEC-01	Auth/API	TC-SEC-01	N
ML-EVAL-SR-001	TLS 1.2+ / HTTPS	5.1 / DS-SEC-02	Deployment/API	TC-SEC-02	N
ML-EVAL-SR-003	Secure upload validation	5.1 / DS-SEC-04	DatasetModule	TC-SEC-04	N
ML-EVAL-SR-005	No arbitrary code execution	5.1 / DS-SEC-06	ML/DataPipeline	TC-SEC-06	N

TEAM 16

NAME	SRN
VASANTH KUMAR REDDY	PES2UG25CS829
MUNIREDDY S	PES2UG25CS813
SHARATH H M	PES2UG25CS822
MANOJ M S	PES2UG25CS810